

Deep Learning Applications In Medical Image Analysis-Brain Tumor

A Real-Time Research Project Report submitted to
JNTU Hyderabad in partial fulfillment
of the requirements for the award of the degree

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

Submitted by

NANDIKONDA LIKHITH REDDY	21S11A6687
DEVADIGA ANIKETH	21S11A6671
THANUGULA MANOHAR	21S11A6689
ADEM VINAY	21S11A66C5

Under the Guidance of
Dr. VAKA MURALI MOHAN
B. Tech, M. Tech, Ph.D.

Principal & Professor of CSE



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

MALLA REDDY INSTITUTE OF TECHNOLOGY AND SCIENCE
UGC Autonomous Institution

Approved by AICTE New Delhi and Affiliated to JNTU
An ISO 9001: 2015 Certified Institution
Accredited by NBA and NAAC 'A' Grade
Maisammaguda, Medchal (Dist), Hyderabad -500100,
Telangana

JUNE-2024

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

MALLA REDDY INSTITUTE OF TECHNOLOGY AND SCIENCE
UGC Autonomous Institution

Approved by AICTE New Delhi and Affiliated to JNTU
An ISO 9001: 2015 Certified Institution
Accredited by NBA and NAAC 'A' Grade
Maisammaguda, Medchal (Dist), Hyderabad -500100, Telangana.

JUNE-2024



CERTIFICATE

This is to certify that the mini project entitled **“Deep Learning Applications In Medical Image Analysis-Brain Tumor”** has been submitted by **NANDIKONDA LIKHITH REDDY (21S11A6687)**, **DEVADIGA ANIKETH (21S11A6671)**, **THANUGULA MANOHAR (21S11A6689)** and **ADEM VINAY (21S11A66C5)** in partial fulfillment of the requirements for the award of **BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING (AI & ML)**. This record of bonafide work carried out by them under my guidance and supervision. **The result embodied in this mini project report has not been submitted to any other University or Institute for the award of any degree.**

Dr. VAKA MURALI MOHAN
Principal & Professor of CSE
Project Guide

Dr. A. NAGARAJU
Head of the Department
CSE (AI&ML)

External Examiner

ACKNOWLEDGEMENT

The Mini Project work carried out by our team in the Department of Computer Science and Engineering, Malla Reddy Institute of Technology and Science, Hyderabad. ***This work is original and has not been submitted in part or full for any degree or diploma of any other university.***

We wish to acknowledge our sincere thanks to our project guide **Dr. Vaka Murali Mohan**, Principal & Professor of Computer Science & Engineering for formulation of the problem, analysis, guidance and her continuous supervision during the course of work.

We acknowledge our sincere thanks to **Dr. Vaka Murali Mohan**, Principal and **Dr. A. Nagaraju**, Head of the Department and Coordinator, faculty members of CSE (AI & ML) Department for their kind cooperation in making this Mini Project work a success.

We extend our gratitude to **Sri. Ch. Malla Reddy**, Founder Chairman MRGI and **Sri. Ch. Mahender Reddy**, Secretary MRGI, **Dr.Ch. Bhadra Reddy**, President MRGI, **Sri. Ch. Shalini Reddy**, Director MRGI, **Sri. P. Praveen Reddy**, Director MRGI, for their kind cooperation in providing the infrastructure for completion of our Mini Project.

We acknowledge our special thanks to the entire teaching faculty and non-teaching staff members of the Computer Science & Engineering Department for their support in making this project work a success.

NANDIKONDA LIKHITH REDDY	21S11A6687
DEVADIGA ANIKETH	21S11A6671
THANUGULA MANOHAR	21S11A6689
ADEM VINAY	21S11A66C5

INDEX

Chapter	Page No.
ABSTRACT	vi
LIST OF FIGURES	vii
1. SYSTEM ANALYSIS	1
1.1 Existing System	1
1.1.1 Disadvantages of Existing System	1
1.2 Proposed System	1
1.2.1 Advantages of Proposed System	2
1.3 Introduction	3
2. LITERATURE SURVEY	4
3. SYSTEM DESIGN	10
3.1 System Architecture	10
3.1.1 Modules	10
3.1.2 Block Diagram	11
3.2 System Requirements	13
3.2.1 Hardware Requirements	13
3.2.2 Software Requirements	13
4. INPUT & OUTPUT DESIGN	14
4.1 Input Design	14
4.2 Output Design	14
5. SOFTWARE ENVIRONMENT	16
5.1 Python Technology	16
5.1.1 History of python	16
5.1.2 What can python do	17
5.1.3 Why python	17
5.1.4 Python syntax compared to other programming languages	18
5.1.5 Uses of Python	18
5.1.6 Python Features	19
5.2 SDLC	20
5.2.1 Benefits of SDLC	20
5.3 Natural Language Processing (NLP)	22

5.4 Similarity Measures	23
6. SYSTEM STUDY	26
6.1 Economic Feasibility	26
6.2 Technical Feasibility	26
6.3 Social Feasibility	26
7. SYSTEM TESTING	27
7.1 Types of Tests	27
7.1.1 Unit testing	27
7.1.2 Integration testing	27
7.1.3 Functional testing	27
7.1.4 System testing	28
7.1.5 Acceptance testing	28
8. SOURCE CODE	29
9. RESULTS	36
10. CONCLUSION & FUTURE ENHANCEMENT	42
11. REFERENCE	45
12. YUKTI INNOVATION CERTIFICATE	

ABSTRACT

This project deals with investigations on neurological disorders in brain. The neurological disorder in the brain has been assessed by unsupervised machine learning and 3D-CNN methods. Unsupervised machine learning and 3D-CNN play a vital role to detect the neurological disorders in brain with best accuracy. So, this project can be implemented by using unsupervised machine learning and 3D-CNN methods. Brain is the controlling center of our body. With advent of time, newer and newer brain diseases are being discovered. Detection of brain diseases at an early stage can make a huge difference in attempting to cure them. In this study, we represent a review on recent machine learning approaches in detecting four brain diseases such as brain tumor and epilepsy. This abstract reviews recent advancements in machine learning applied to neurology, emphasizing the potential of innovative computational approaches in revolutionizing medical diagnostics and patient care.

LIST OF FIGURES

Fig.No.	Fig. Name	Page No
Chapter:3-Figure 1	Block Diagram	11
Chapter:3-Figure 2	Data Flow Diagram	12
Chapter:3-Figure 3	Activity Diagram	12
Chapter:8-Figure 4	Opening the project	36
Chapter:8-Figure 5	Run the Project	36
Chapter:8-Figure 6	GUI of the Project	37
Chapter:8-Figure 7	Upload MRI Images Dataset	37
Chapter:8-Figure 8	Uploaded Dataset Screen	38
Chapter:8-Figure 9	Generate Images Train & Test Model	38
Chapter:8-Figure 10	Display of Classes and Images	39
Chapter:8-Figure 11	Generate Deep Learning CNN Model	39
Chapter:8-Figure 12	Output of Model Training 1	40
Chapter:8-Figure 13	Output of Model Training 2	40
Chapter:8-Figure 14	Test the Model	41
Chapter:8-Figure 15	Output Screen of Brain Tumor	41

CHAPTER-1: SYSTEM ANALYSIS

1.1 Existing System

There is a myriad of imaging modalities, and the frequency of their use is increasing. Smith-Bindman *et al.* [2] looked at imaging use from 1996 to 2010 across six large integrated healthcare systems in the United States, involving 30.9 million imaging examinations. The authors found that over the study period, CT, MRI and PET usage increased 7.8%, 10% and 57% respectively.

1.1.1 Disadvantages of Existing System

The escalation in the utilization of imaging technologies, as documented by Smith-Bindman *et al.*, introduces several drawbacks. The heightened reliance on CT, MRI, and PET scans can lead to overutilization, where unnecessary scans are performed, increasing healthcare costs and exposing patients to potential harm, such as radiation-induced risks from CT scans. This trend can also contribute to a diagnostic cascade, where incidental findings trigger further tests and interventions, often without clear clinical benefit. The complexity of these imaging techniques requires highly skilled professionals, potentially leading to disparities in access to quality care. Moreover, the environmental impact of producing and disposing of imaging equipment, along with the energy consumption associated with their operation, raises sustainability concerns. These factors underscore the need for careful oversight and judicious use of imaging technologies to mitigate these disadvantages.

1.2 Proposed System

Our proposed system demonstrates a block design using an X-ray/CT scan image as an input from a dataset split into 2 sections: brain cancer patients and healthy people. Before preparing and surveying the information, preprocessing is performed. The image aspects are resized, the picture pictures are changed over to exhibits, the info is preprocessed utilizing MobileNetV2, and the last advance in every one of the four preprocessing steps is the encoding of hot marks. Scaling a picture is a significant preprocessing step in PC vision since it influences the presentation of the preparation model. It performs all the better when the picture is more modest. The image was extended to 256×256 pixels for this examination. The following stage is to change the photos in the assortment into a cluster. The image is changed over to a cluster

prior to being utilized on the up and up work. MobileNetV2 involves the image as a pre technical input. The last advance is to do hot coding on marks since many AI calculations cannot work straightforwardly on named information. This calculation's feedback and result factors should be numeric. For understanding and examination, the labeled information is converted into a mathematical mark. In the wake of preprocessing, the information is parceled into three areas: 70% preparation information, 20% approval information, and excess testing information. Individuals who are solid as well as those who have been determined to have a brain disease are displayed in each heap of photographs.

Epilepsy is one of the most common brain disorders worldwide. The most frequently used clinical tool to detect epileptic events and monitor epilepsy patients is the EEG recordings. There have been proposed many computer-aided diagnosis systems using EEG signals for the detection and prediction of seizures. In this study, a novel method based on Convolutional Neural Network (CNN) which is integrated with C-LSTM approach. This is proposed to detect and predict seizure segments.

1.2.1 Advantages of Proposed System

- The proposed system utilizing Convolutional Neural Networks (CNNs) for medical image analysis presents several significant advantages. CNNs are particularly adept at preserving spatial relationships within images, a critical aspect in medical diagnostics where precise localization of anatomical structures or pathological changes is essential. By processing raw pixel data through convolutional layers, CNNs can extract intricate features that reflect the complex spatial arrangements seen in radiological images, such as the junction between bone and muscle or the differentiation between healthy and cancerous lung tissues. The integration of Rectified Linear Unit (ReLU) layers enhances the network's ability to learn non-linear patterns, while pooling layers reduce dimensionality, improving computational efficiency and reducing overfitting. The final fully connected layer of the CNN provides a robust mechanism for classification, assigning high probabilities to the most likely diagnoses based on the learned features. This automated and accurate classification system not only accelerates the diagnostic process but also enhances the consistency and objectivity of medical image interpretation, potentially leading to improved patient outcomes and more efficient healthcare delivery.

1.3 Introduction

Machine learning algorithms have the potential to be invested deeply in all fields of medicine, from drug discovery to clinical decision making, significantly altering the way medicine is practiced. The success of machine learning algorithms at computer vision tasks in recent years comes at an opportune time when medical records are increasingly digitalized. The use of electronic health records (EHR) quadrupled from 11.8% to 39.6% amongst office-based physicians in the US from 2007 to 2012 [1]. Medical images are an integral part of a patient's EHR and are currently analyzed by human radiologists, who are limited by speed, fatigue, and experience. It takes years and great financial cost to train a qualified radiologist, and some health-care systems outsource radiology reporting to lower-cost countries such as India via tele-radiology. A delayed or erroneous diagnosis causes harm to the patient. Therefore, it is ideal for medical image analysis to be carried out by an automated, accurate and efficient machine learning algorithm. Medical image analysis is an active field of research for machine learning, partly because the data is relatively structured and labelled, and it is likely that this will be the area where patients first interact with functioning, practical artificial intelligence systems. This is significant for two reasons. Firstly, in terms of actual patient metrics, medical image analysis is a litmus test as to whether artificial intelligence systems will actually improve patient outcomes and survival. Secondly, it provides a testbed for human-AI interaction, of how receptive patients will be towards healthaltering choices being made, or assisted by a non-human actor.

CHAPTER-2: LITERATURE SURVEY

Title 1: A deep learning approach for prediction of Parkinson's disease progression

Authors: Afzal Hussain Shahid¹ · Maheshwari Prasad Singh

Description: This paper proposes a deep neural network (DNN) model using the reduced input feature space of Parkinson's telemonitoring dataset to predict Parkinson's disease (PD) progression. PD is a chronic and progressive nervous system disorder that affects body movement. PD is assessed by using the unified Parkinson's disease rating scale (UPDRS). In this paper, firstly, principal component analysis (PCA) is employed to the featured dataset to address the multicollinearity problems in the dataset and to reduce the dimension of input feature space. Then, the reduced input feature space is fed into the proposed DNN model with a tuned parameter norm penalty (L2) and analyses the prediction performance of it in PD progression by predicting Motor and Total-UPDRS score. The model's performance is evaluated by conducting several experiments and the result is compared with the result of previously developed methods on the same dataset. The model's prediction accuracy is measured by fitness parameters, mean absolute error (MAE), root mean squared error (RMSE), and coefficient of determination (R²). The MAE, RMSE, and R² values are 0.926, 1.422, and 0.970 respectively for motor-UPDRS. These values are 1.334, 2.221, and 0.956 respectively for Total-UPDRS. Both the Motor and Total-UPDRS score is better predicted by the proposed method. This paper shows the usefulness and efficacy of the proposed method for predicting the UPDRS score in PD progression. This paper proposed a PCA based DNN model for the prediction of Motor-UPDRS and Total-UPDRS in PD progression. The DNN model was evaluated on a real-world PD dataset taken from UCI. It was observed that the proposed DNN model may precisely capture non-linearity in the feature space and shows better performance than the previously known methods, namely MLR, NN, ANFIS, SVR, HSLSSVR, and SOM-NIPALS-ISVR. Being a DNN model, the performance of the proposed model may improve with the addition of more data points in the datasets. PCA was used to extract the features from the whole dataset that include age and gender of the PD patients. Clinically, the age and gender affect the UPDRS score. Therefore, the proposed method is better, reliable, and robust in predicting the UPDRS score.

Title 2: A Novel MRI Diagnosis Method for Brain Tumor Classification Based on CNN and Bayesian Optimization

Author: Mohamed Ait Amou, Kewen Xi , Souha Kamhi and Mohamed Mouhafid

Description: Brain tumor is one of the most aggressive diseases nowadays, resulting in a very short life span if it is diagnosed at an advanced stage. The treatment planning phase is thus essential for enhancing the quality of life for patients. The use of Magnetic Resonance Imaging (MRI) in the diagnosis of brain tumors is extremely widespread, but the manual interpretation of large amounts of images requires considerable effort and is prone to human errors. Hence, an automated method is necessary to identify the most common brain tumors. Convolutional Neural Network (CNN) architectures are successful in image classification due to their high layer count, which enables them to conceive the features effectively on their own. The tuning of CNN hyperparameters is critical in every dataset since it has a significant impact on the efficiency of the training model. Given the high dimensionality and complexity of the data, manual hyperparameter tuning would take an inordinate amount of time, with the possibility of failing to identify the optimal hyperparameters. In this paper, we proposed a Bayesian Optimization-based efficient hyperparameter optimization technique for CNN. This method was evaluated by classifying 3064 T-1-weighted CE-MRI images into three types of brain tumors (Glioma, Meningioma, and Pituitary). Based on Transfer Learning, the performance of five well-recognized deep pre-trained models is compared with that of the optimized CNN. After using Bayesian Optimization, our CNN was able to attain 98.70% validation accuracy at best without data augmentation or cropping lesion techniques, while VGG16, VGG19, ResNet50, InceptionV3, and DenseNet201 achieved 97.08%, 96.43%, 89.29%, 92.86%, and 94.81% validation accuracy, respectively. Moreover, the proposed model outperforms state-of-the-art methods on the CE-MRI dataset, demonstrating the feasibility of automating hyperparameter optimization. Brain tumor classification is among the most crucial aspects of the medical field. Building an efficient CNN is not an easy task. For this reason, it has become essential to use optimization methods to set CNN hyperparameters. This paper proposed a new approach that classifies among three common brain tumor classes (Glioma, Meningioma, and Pituitary). The optimal hyper parameters values of the model are selected using Bayesian Optimization technique. On the other hand, five pre-trained CNNs were fine-tuned and trained on the same dataset. Research findings indicated that the optimized model gave the best classification performance with 98.70% accuracy followed by VGG16 with 97.08% accuracy whereas ResNet50 attained the lowest results with 89.29% accuracy. It is concluded that automating hyper parameter optimization is effective in increasing the performance of a scratched CNN model. Training and validation are conducted using MRI scans in axial, coronal, and sagittal planes. The model presented in this paper can be extended to classify other diseases effectively

Title 3: An Efficient Classifier for Alzheimer’s disease Genes Identification

Author: Lei Xu 1 , Guangmin Liang 1 , Changrui Liao 2 , Gin-Den Chen 3,* and Chi-Chang Chang 4,5,*

Description: Alzheimer’s disease (AD) is considered to one of 10 key diseases leading to death in humans. AD is considered the main cause of brain degeneration, and will lead to dementia. It is beneficial for affected patients to be diagnosed with the disease at an early stage so that efforts to manage the patient can begin as soon as possible. Most existing protocols diagnose AD by way of magnetic resonance imaging (MRI). However, because the size of the images produced is large, existing techniques that employ MRI technology are expensive and time-consuming to perform. With this in mind, in the current study, AD is predicted instead by the use of a support vector machine (SVM) method based on gene-coding protein sequence information. In our proposed method, the frequency of two consecutive amino acids is used to describe the sequence information. The accuracy of the proposed method for identifying AD is 85.7%, which is demonstrated by the obtained experimental results. The experimental results also show that the sequence information of gene-coding proteins can be used to predict AD. In this paper, a computational method based on protein sequence information was introduced to predict the onset of AD. In our proposed method, the sequences are represented by the frequency of two consecutive amino acids, and then the data are classified by SVM. Our work is different from previous work that was completed using MRI, which is time-consuming and expensive. As demonstrated by the presented experimental results, the classification accuracy of our proposed method is 85.7%. Moreover, a dataset used for AD classification was created in our work. In future work, we will try to mine the relationships between the features to improve the classification performance of the predictions method. Furthermore, due to the wide use of webservers in bioinformatics, such as the work of RNA secondary structure comparison [73], we will also develop the a webserver for AD prediction.

Title 4: A Machine Learning Based Approach for Early Detection of Alzheimer’s Disease by Extracting Texture and Shape Features of the Hippocampus Region from MRI Scans

Author: Arpita Raut1 , Vipul Dalal2

Description: Alzheimer’s Disease (AD) is a neurological disease and the most common cause of dementia in the age group 65 years and above. The accurate and timely diagnosis of the AD is crucial in order to prevent the progression of this irreversible disease. This paper concentrates on a method to detect Alzheimer’s from MRI scans using machine learning approach. The proposed approach focuses on the hippocampal region of the brain. The texture features such

as entropy, homogeneity, energy, contrast, correlation and variance are extracted from the hippocampus region using the Gray Level Co-Occurrence Matrix (GLCM). The area and shape feature are extracted using the Moment Invariants. The Error-back Propagation (EBP) in Artificial Neural Network (ANN) is used as the classifier for detection of various stages of AD. The proposed system gives an average accuracy of 86.8%. In this work, a machine learning based approach for detection of Alzheimer's disease is proposed. The OASIS dataset was used for experiments. The texture, area and shape features from hippocampus region of MRI scan are extracted. The textual features from OASIS were also extracted. These features were used to train the neural network with errorback propagation for classification. The proposed system has an average accuracy of 86.8 %.

Title 5: Automated detection of Alzheimer's Disease using Deep Learning in MRI

Author: P C Muhammed Raees and Vinu Thomas

Description: Alzheimer's disease (AD) is a progressive mental deterioration and incurable neurodegenerative disease that can occur in middle or old age, due to generalized degeneration of the brain. Because of the irreversible nature of the progression of Alzheimer's disease, the early diagnosis of AD has an immense clinical, social, and economic need. This research output proposing a state-of-the-art, easy, and early automated deep learning-based system to predict AD from a large MRI dataset of normal and diseased subjects. It classified the database of 111 subjects into Mild Cognitive Impairment (MCI), Alzheimer's disease (AD), and Normal classes. Classification tools like Support Vector Machines (SVM) and different models of Deep Neural Network (DNN) algorithms were tested. Deep learning algorithms were offering high accuracy of about 80-90% on AD prediction. For the prediction of diseases such as Alzheimer's, Dementia, and Parkinson's, applying highly accurate computational - automated machine learning - tools will help to diagnose the disease in the early stage itself and provide a better clinical, social and economic outcome. An automated machine learning tool for the prediction of Alzheimer's disease using a deep learning algorithm has been successfully designed and implemented by this work. The performance levels of SVM and DNN models were also examined. Deep learning shows high accuracy level of about 80-90% in Alzheimer's disease prediction. The method suggested in this work will solve social and economic problems of AD, if it is successfully implemented in clinical practices. At present, the clinical trials of computational techniques are very limited. For the prediction of diseases such as Alzheimer's, Dementia, and Parkinson's applying highly accurate computational - automated machine

learning - tools will help to diagnose the disease in the early stage itself and reduce the treatment time.

Title 6: Accurate Blood-Based Diagnostic Bio signatures for Alzheimer's disease via Automated Machine Learning

Author: Makrina Karagiani, Krystallia Gourlia, Ioannis Tsamardinos and Ekaterini Chatzaki

Description: Alzheimer's disease (AD) is the most common form of neurodegenerative dementia and its timely diagnosis remains a major challenge in biomarker discovery. In the present study, we analyzed publicly available high-throughput low-sample -omics datasets from studies in AD blood, by the Auto ML technology Just Add Data Bio (JADBIO), to construct accurate predictive models for use as diagnostic bio signatures. Considering data from AD patients and age–sex matched cognitively healthy individuals, we produced three best performing diagnostic bio signatures specific for the presence of AD: A. A 506-feature transcriptomic dataset from 48 AD and 22 controls led to a miRNA-based bio signature via Support Vector Machines with three miRNA predictors (AUC 0.975 (0.906, 1.000)), B. A 38,327-feature transcriptomic dataset from 134 AD and 100 controls led to six mRNA-based statistically equivalent signatures via Classification Random Forests with 25 mRNA predictors (AUC 0.846 (0.778, 0.905)) and C. A 9483-feature proteomic dataset from 25 AD and 37 controls led to a protein-based biosignature via Ridge Logistic Regression with seven protein predictors (AUC 0.921 (0.849, 0.972)). These performance metrics were also validated through the JADBIO pipeline confirming stability. In conclusion, using the automated machine learning tool JADBIO, we produced accurate predictive biosignatures extrapolating available low sample -omics data. These results offer options for minimally invasive blood-based diagnostic tests for AD, awaiting clinical validation based on respective laboratory assays. They also highlight the value of AutoML in biomarker discovery. In conclusion, using the autoML JADBIO tool, we produced specific predictive biosignatures extrapolating available low sample multi-omics data. These results offer options for minimally invasive blood-based diagnostic tests, awaiting clinical validation based on respective laboratory assays for the classification of Alzheimer's disease. They also highlight the value of autoML in biomarker discovery. All three reported biosignatures showing high performance and $AUC > 0.85$ are worthy of further validation. Still, if we were to choose one as the most promising, we would propose this to be the miRNA-transcriptomic biosignature based on three apparent advantages: highest performance [AUC 0.975 (0.906, 1.000)], fewer features (three predictors), entities allowing easy multiplexing assays (miRNA). A side-conclusion of this study yet quite

important, is the following: revisiting and reanalyzing “old” datasets is not only potentially fruitful but scientifically necessary. The current mentality of life-scientists is often to discard public data as “used”, under the assumption that all discoveries to be made, are already published. However, as new and powerful statistical and computational methods are introduced, such as JADBIO and AutoML, new types of analyses become possible and new patterns and results become ripe for discovery. It is not only computational methods that change; the analysis context is constantly changing. It is thus worth re-analyzing past datasets in the context of new datasets submitted recent scientific publications, or newly added or revised knowledge in biological databases (e.g., updated pathways).

CHAPTER-3: SYSTEM DESIGN

3.1 System Architecture

3.1.1 Modules

- **Upload MRI image**

Using this module we are uploading MRI train images and then application read all images and convert them grey format.

Generate Train & Test Model

- Using this module we will build array of pixels with all images features and then split dataset into train and test model to calculate accuracy using test images by applying train model on it.

Generate Deep Learning CNN Model

- Using this module will input train and test data to auto stack CNN model to build training classifier.

Get DriveHQ Images

- Using this module we will read test image from DriveHQ website and then application will apply CNN classifier model on that test image to predict whether image contains tumour disease or not.

3.1.2 Block Diagram

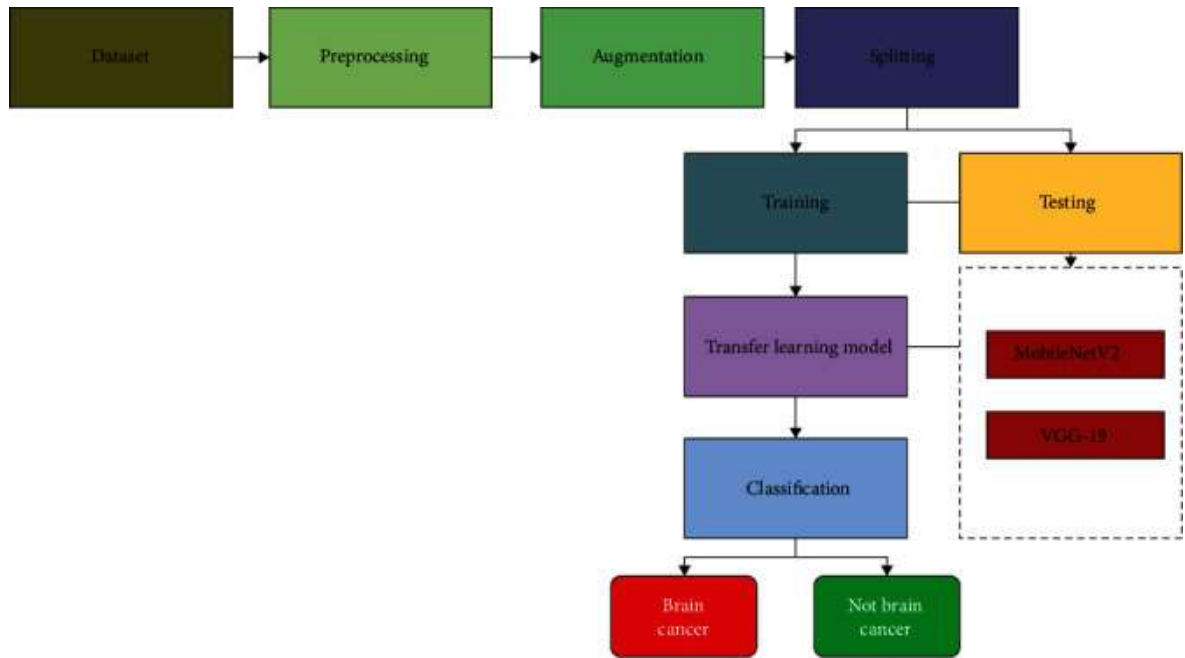


Figure 1: Block Diagram

❖ DESIGN METHODOLOGY

Transfer Learning with MobileNetV2: Clinical scientists face a few obstructions because of an absence of clinical information or datasets. Deep learning strategies are very information-subordinate. Information examination and characterization is a tedious and costly interaction. The advantage of machine learning is that it does not need to bother with an enormous dataset. Estimations become less difficult and more expensive as time passes. Move learning is a technique that involves moving the data acquired from a prepared model on an enormous dataset to another model that needs substantially less information to be prepared. This method began with CNN preparing a little dataset for a particular reason, which incorporated a huge scope dataset that had been recently prepared to utilize prepared models [19]. Three prepared CNN-based models were utilized to sort mind X-beam pictures in this review. MobileNetV2, VGG19, and Inception V3 models were utilized. Two kinds of mind X-beam pictures exist: the left one is ordinary, yet the right one has grown. Moreover, this examination utilized an exchange learning approach that is equipped to perform admirably with meager information and is likewise time-effective by utilizing ImageNet information. The system architecture of the transfer learning approach is represented in Figure 3.

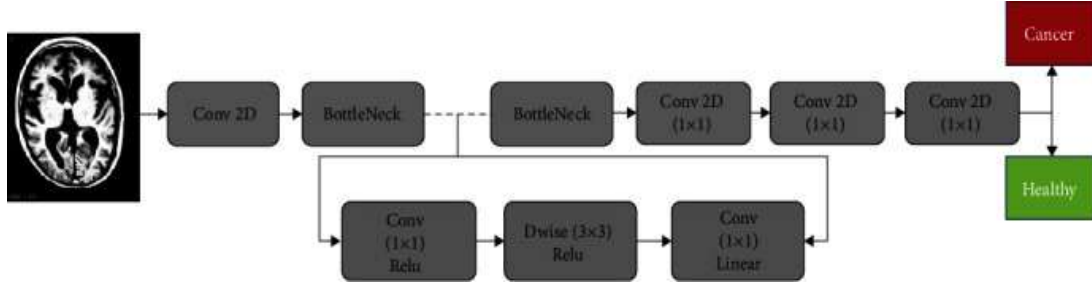


Figure 2: Data Flow Diagram

Figure 3 is broken into four significant areas. The primary area gives X-beam photos of the cerebrum. The subsequent part demonstrates how to stack a preprepared model. Three preprepared models are remembered for the subsequent stage. The last stage, as demonstrated in Figure 3, adds the accompanying layers to the stacked, preprepared models. Finally, the result segment isolates the discoveries into two classifications: harmful growth and solid growth. Experimental results demonstrate that the proposed approach yields high average segment-based seizure detection precision and accuracy rates for both datasets (ACC; CHB-MIT: 93.81%).

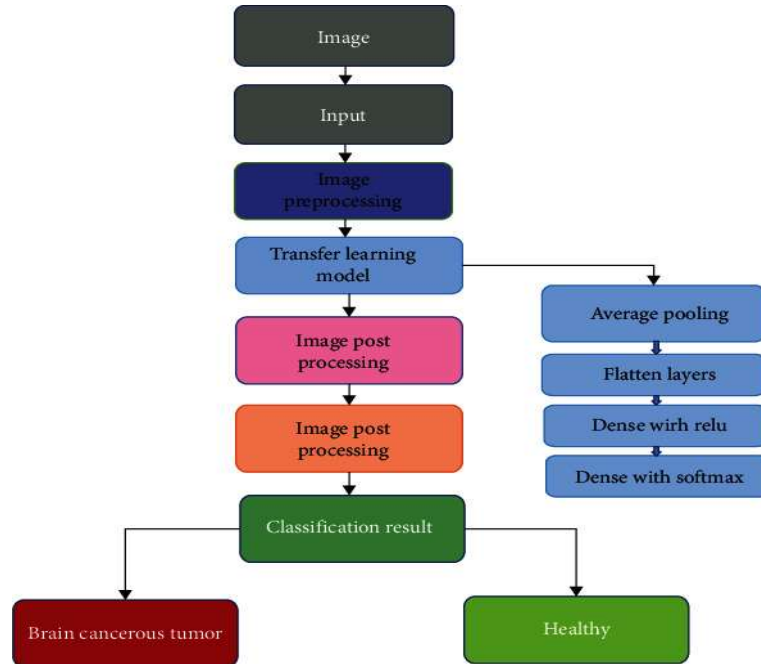


Figure 3: Activity Diagram

To overcome the problems of automated brain tumor classification, a novel approach is proposed based on long short-term memory (C-LSTM) model using magnetic resonance images (MRI). First, N4ITK and Gaussian filters having size 5×5 are used to boost the of multi-sequence MRI quality. The presented deep LSTM model having four layers is utilized

for classification. In each layer, optimal hidden units (HU) are selected such as 200 HU, 225 HU, 200 HU and 225 HU, respectively.

3.2 System Requirements

A Software requirements specification (SRS) document describes the intended purpose, requirements, and nature of software/application/project to be developed. To prepare an SRS document, you would need to have a functional knowledge of your project or application, knowledge of software/hardware/technology to be used.

3.2.1 Hardware Requirements

- System Processor : Pentium IV 2.4 GHz or above
- Hard Disk : 20 GB or above
- Monitor : 10/12/15 VGA color
- Mouse : Three button windows standard mouse
- Keyboard : Windows standard keyboard.
- Ram : 4GB or above

3.2.2 Software Requirements

- Operating system : Windows 7/8/10/11.
- Programming Language : python 3.7
- IDE : PyCharm Community Edition
- Type of Application : Graphical User Interface
- Frontend Technologies : Tkinter, API, CNN & LSTM with supported libraries
- Backend Technologies : matplotlib, TensorFlow, Keras, etc.
- Dataset : X-ray/CT scan image (Brain)

CHAPTER-4: INPUT AND OUTPUT DESIGN

4.1 Input Design

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy. Input Design considered the following things:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

OBJECTIVES

1. Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.

2. It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities.

3. When the data is entered it will check for its validity. Data can be entered with the help of screens. Appropriate messages are provided as when needed so that the user will not be in maize of instant. Thus the objective of input design is to create an input layout that is easy to follow

4.2 Output Design

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

1. Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.

2. Select methods for presenting information.

3. Create document, report, or other formats that contain information produced by the system.

The output form of an information system should accomplish one or more of the following objectives.

- Convey information about past activities, current status or projections of the
- Future.
- Signal important events, opportunities, problems, or warnings.
- Trigger an action.
- Confirm an action.

CHAPTER-5: SYSTEM ENVIRONMENT

5.1 PYTHON

Python is a **high-level, interpreted, interactive and object-oriented scripting language**. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted:** Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive:** You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented:** Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language:** Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

5.1.1 History of Python

Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.

Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell and other scripting languages.

Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL).

Python is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

5.1.2 What can python do

Python is a versatile and powerful programming language that can accomplish a wide array of tasks, making it a popular choice across various domains. In the realm of software development, Python can be used to build web applications, desktop software, and network servers, leveraging frameworks like Django and Flask for web development. Its simplicity and readability make it an excellent language for rapid prototyping and scripting, enabling developers to automate repetitive tasks and streamline workflows. Python's extensive standard library and numerous third-party packages support data analysis and scientific computing, with tools like NumPy, pandas, and Matplotlib facilitating complex data manipulation and visualization. In the field of artificial intelligence and machine learning, Python is the language of choice, with libraries such as TensorFlow, PyTorch, and scikit-learn providing robust tools for building and deploying sophisticated models. Additionally, Python's support for cross-platform compatibility ensures that applications and scripts can run on various operating systems without significant modifications. Overall, Python's flexibility, ease of use, and strong community support make it a go-to language for a diverse range of programming tasks and projects.

5.1.3 Why python

Python's widespread adoption and enduring popularity can be attributed to several key factors that make it an exceptional choice for both novice and experienced programmers. One of the primary reasons for Python's appeal is its simplicity and readability. The language's syntax is designed to be intuitive and clear, resembling natural language, which simplifies the learning curve and enhances code maintainability. This readability also fosters collaboration, as it allows developers to easily understand and contribute to each other's codebases.

Another significant advantage of Python is its extensive standard library and the vast ecosystem of third-party packages. This wealth of resources covers a multitude of functionalities, from web development and data analysis to machine learning and automation, enabling developers to leverage pre-built tools and modules to expedite development and reduce the need for reinventing the wheel.

Python's versatility is another compelling reason for its popularity. It is a general-purpose language that can be applied to a wide range of applications, from small scripts to large-scale

enterprise systems. Its support for multiple programming paradigms, including procedural, object-oriented, and functional programming, allows developers to choose the most appropriate approach for their projects.

5.1.4 Python syntax compared to other programming languages

Python's syntax stands out among programming languages for its emphasis on simplicity and readability. Unlike languages such as C++ or Java, which often require verbose code and explicit type declarations, Python adopts a minimalist approach with significant use of whitespace to delimit blocks of code, eliminating the need for braces or semicolons that can clutter the code in other languages. This indentation-based syntax not only makes the code visually cleaner but also enforces a clear structure that can reduce errors and enhance readability.

Python's syntax is designed to be intuitive, with a focus on natural language expressions. For example, its use of English keywords like 'and', 'or', and 'not' instead of symbols makes the code more approachable for beginners and easier to understand for all developers. Additionally, Python's dynamic typing allows for more flexible coding, as variables do not need to be explicitly declared with a type, which can speed up development and reduce boilerplate code.

In contrast to more rigid languages, Python's syntax encourages modularity and code reuse through its support for high-level data types and operations, such as list comprehensions and dictionary manipulations, which can replace multiple lines of code in other languages with a single, concise expression. This conciseness not only saves time but also reduces the potential for errors.

Moreover, Python's extensive use of libraries and modules, which are imported with simple and consistent syntax, facilitates the integration of complex functionalities with minimal code. This ease of use and integration is a significant factor in Python's popularity across various domains, from web development to scientific computing and machine learning.

5.1.5 Uses of Python

Python is renowned for its versatility and is employed across a broad spectrum of applications. In the realm of web development, Python powers dynamic websites and web applications through frameworks like Django and Flask, offering robust tools for backend development, API creation, and database management. Its scripting capabilities make it indispensable for automating routine tasks, from file manipulation to system administration, enhancing productivity and efficiency.

In the scientific and numerical computing domain, Python is a go-to language, thanks to libraries such as NumPy, pandas, and SciPy, which facilitate data analysis, statistical modeling, and numerical computations. Its visualization tools, like Matplotlib and seaborn, enable the creation of insightful data visualizations, aiding in research and decision-making processes.

Python's prowess extends to artificial intelligence and machine learning, where libraries like TensorFlow, PyTorch, and scikit-learn are used to build, train, and deploy complex models for tasks such as image recognition, natural language processing, and predictive analytics. Its simplicity and flexibility make it an ideal language for rapid prototyping and experimentation in these fields.

Additionally, Python is widely used in education and academia for teaching programming concepts due to its clear syntax and ease of learning. It also plays a crucial role in the finance industry, where it is used for quantitative analysis, algorithmic trading, and risk management.

5.1.6 Python Features

Python's features include:

- **Easy-to-learn:** Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read:** Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain:** Python's source code is fairly easy-to-maintain.
- **A broad standard library:** Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.

- **Interactive Mode:** Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable:** Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable:** You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases:** Python provides interfaces to all major commercial databases.
- **GUI Programming:** Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable:** Python provides a better structure and support for large programs than shell scripting.

Python has a big list of good features:

- It supports functional and structured programming methods as well as OOP.
- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.
- IT supports automatic garbage collection.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.

5.2 SDLC

5.2.1 Benefits of SDLC

The Software Development Life Cycle (SDLC) is a structured process that guides the development of software from inception to retirement. It encompasses various phases, including planning, design, development, testing, deployment, and maintenance. The benefits of adhering to an SDLC approach are numerous and contribute to the successful delivery and management of software projects. Here are some key advantages:

Improved Planning and Organization: SDLC provides a systematic framework that helps in defining project scope, objectives, and deliverables. This structured approach ensures that all stakeholders have a clear understanding of the project's goals and the steps required to achieve them.

Enhanced Quality and Reliability: By incorporating rigorous testing phases, SDLC ensures that software is thoroughly checked for bugs and errors before it is released. This leads to higher quality products that are more reliable and perform better in real-world scenarios.

Increased Efficiency and Productivity: The SDLC's phased approach allows for better resource allocation and task management. It helps in identifying bottlenecks and inefficiencies early in the process, enabling teams to optimize their workflows and improve productivity.

Better Risk Management: SDLC facilitates the early identification and mitigation of risks through regular reviews and checkpoints. This proactive approach helps in minimizing the impact of potential issues and ensures that the project stays on track.

Enhanced Collaboration and Communication: The structured nature of SDLC promotes clear communication among team members, stakeholders, and clients. It provides a common framework for discussing progress, challenges, and decisions, which leads to better collaboration and alignment.

Cost-Effectiveness: By planning and managing resources effectively, SDLC helps in controlling costs. It ensures that the project stays within budget by preventing scope creep and reducing the need for rework due to undetected issues.

Scalability and Flexibility: SDLC methodologies, such as Agile and DevOps, emphasize iterative development and continuous improvement. This allows for greater flexibility in adapting to changing requirements and scaling the project as needed.

Documentation and Knowledge Transfer: SDLC emphasizes the importance of documentation at each phase. This ensures that knowledge is captured and can be easily

transferred, facilitating onboarding of new team members and providing a reference for future projects.

Customer Satisfaction: By delivering high-quality software that meets customer needs and expectations, SDLC contributes to higher customer satisfaction. The iterative and customer-centric approaches ensure that feedback is incorporated throughout the development process.

Lifecycle Management: SDLC extends beyond the initial development phase to include maintenance and updates. This ensures that the software remains relevant, secure, and functional over its entire lifecycle, providing long-term value to the organization.

5.3 Natural Language Processing (NLP)

Natural Language Processing (NLP) is a branch of artificial intelligence (AI) that focuses on the interaction between computers and humans through natural language. The ultimate objective of NLP is to enable computers to understand, interpret, and respond to human language in a way that is both meaningful and contextually appropriate. NLP combines computational linguistics—rule-based modeling of human language—with statistical, machine learning, and deep learning models to process and analyze large volumes of natural language data.

Key components and applications of NLP include:

Language Modeling: This involves creating models that can predict the next word or phrase in a sequence, which is fundamental for tasks like speech recognition, machine translation, and text generation.

Speech Recognition: NLP techniques are used to convert spoken language into text, enabling applications like virtual assistants (e.g., Siri, Alexa) and dictation software.

Machine Translation: NLP powers automated translation services that can convert text from one language to another, such as Google Translate, facilitating cross-language communication.

Sentiment Analysis: NLP can analyze text to determine the emotional tone behind words, helping businesses understand customer feedback, social media sentiment, and public opinion.

Text Classification: NLP is used to categorize text into different classes or categories, which is useful for spam filtering, topic labeling, and content recommendation.

Information Extraction: NLP techniques can extract structured information from unstructured text, such as identifying names, dates, and places in news articles or research papers.

Question Answering: NLP systems can parse and understand questions posed in natural language and provide relevant answers, as seen in chatbots and virtual customer service agents.

Summarization: NLP can generate concise summaries of longer documents or articles, saving time for readers and helping to distill key information.

Dialogue Systems: These are conversational agents that use NLP to understand and respond to user inputs, facilitating interactive experiences in applications like customer service and virtual assistants.

Named Entity Recognition (NER): NLP can identify and classify named entities in text into predefined categories such as names of persons, organizations, locations, dates, etc.

NLP technologies are continuously evolving, with advancements in deep learning and neural network architectures leading to significant improvements in the accuracy and sophistication of NLP applications. These developments are paving the way for more natural and intuitive human-computer interactions, enhancing the capabilities of AI systems across various domains.

5.4 Similarity Measures

Similarity measures are mathematical techniques used to quantify how alike two data objects are. In various fields such as data science, machine learning, information retrieval, and natural language processing, similarity measures play a crucial role in clustering, classification, recommendation systems, and pattern recognition. Here are some common types of similarity measures:

Euclidean Distance: This is a straightforward measure that calculates the straight-line distance between two points in a multi-dimensional space. It is widely used in geometry and can be applied to continuous data.

Cosine Similarity: This measure computes the cosine of the angle between two vectors. It is particularly useful in text analysis and recommendation systems, where the magnitude of the vectors is less important than their direction. Cosine similarity ranges from -1 (completely dissimilar) to 1 (completely similar), with 0 indicating orthogonality.

Jaccard Index: Also known as the Jaccard similarity coefficient, this measure is used for comparing the similarity and diversity of sample sets. It is defined as the size of the intersection divided by the size of the union of the sample sets. It is commonly used in data clustering and segmentation tasks.

Pearson Correlation Coefficient: This measure assesses the linear relationship between two continuous variables. It ranges from -1 to 1, where 1 indicates a perfect positive linear relationship, -1 indicates a perfect negative linear relationship, and 0 indicates no linear relationship.

Manhattan Distance: Also known as the city block distance or L1 norm, this measure calculates the distance between two points as the sum of the absolute differences of their coordinates. It is useful in cases where the path between points is constrained, such as in grid-like structures.

Hamming Distance: This measure counts the number of positions at which the corresponding symbols (bits) in two strings of equal length are different. It is commonly used in information theory and coding theory.

Mahalanobis Distance: This measure considers the correlation between variables and is scale-invariant. It is particularly useful when dealing with multivariate data that may have different variances and correlations among variables.

Edit Distance (Levenshtein Distance): This measure calculates the minimum number of single-character edits (insertions, deletions, or substitutions) required to change one word into another. It is used in applications like spell checking and DNA sequence alignment.

Dice's Coefficient: Similar to the Jaccard Index, Dice's Coefficient measures the similarity between two sets and is often used in image segmentation and text mining.

Kernel Functions: These are similarity measures used in machine learning to map data into a higher-dimensional space where it can be more easily separated or clustered. Common kernel functions include the Gaussian (RBF) kernel and polynomial kernel.

The choice of similarity measure depends on the nature of the data and the specific requirements of the application. Proper selection and tuning of these measures can significantly impact the performance of algorithms that rely on them.

CHAPTER-6: SYSTEM STUDY

6.1 Economic Feasibility

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

6.2 Technical Feasibility

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

6.3 Social Feasibility

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

CHAPTER-7: SYSTEM TESTING

7.1 Types of Tests

7.1.1 Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs

7.1.2 Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

7.1.3 Functional testing

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

- Valid Input : identified classes of valid input must be accepted.
- Invalid Input : identified classes of invalid input must be rejected.
- Functions : identified functions must be exercised.
- Output : identified classes of application outputs must be exercised.

Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

7.1.4 System testing

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

7.1.5 Acceptance testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

CHAPTER-8: SOURCE CODE

```
from tkinter import messagebox
from tkinter import *
from tkinter import simpledialog
import tkinter
from tkinter import filedialog
import matplotlib.pyplot as plt
import numpy as np
from tkinter.filedialog import askopenfilename
import os
import cv2
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import imutils
from keras.utils import to_categorical
from keras.layers import MaxPooling2D
from keras.layers import Dense, Dropout, Activation, Flatten
from keras.layers import Convolution2D
from keras.models import Sequential
from keras.models import model_from_json
import pickle
from sklearn import metrics
import ftplib
from tkinter import ttk

main = tkinter.Tk()
main.title("Identifying Brain Tumor using X-Ray Images") #designing main
screen
main.geometry("1300x1200")

global filename
global accuracy
X = []
Y = []
global classifier
disease = ['No Tumor Detected','Tumor Detected']

with open('Model/segmented_model.json', "r") as json_file:
    loaded_model_json = json_file.read()
    segmented_model = model_from_json(loaded_model_json)
json_file.close()
segmented_model.load_weights("Model/segmented_weights.h5")
```

```

segmented_model._make_predict_function()

def edgeDetection():
    img = cv2.imread('myimg.png')
    orig = cv2.imread('test1.png')
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    thresh = cv2.threshold(gray, 30, 255, cv2.THRESH_BINARY)[1]
    contours = cv2.findContours(thresh, cv2.RETR_TREE,
cv2.CHAIN_APPROX_SIMPLE)
    contours = contours[0] if len(contours) == 2 else contours[1]
    min_area = 0.95*180*35
    max_area = 1.05*180*35
    result = orig.copy()
    for c in contours:
        area = cv2.contourArea(c)
        cv2.drawContours(result, [c], -1, (0, 0, 255), 10)
        if area > min_area and area < max_area:
            cv2.drawContours(result, [c], -1, (0, 255, 255), 10)
    return result

def tumorSegmentation(filename):
    global segmented_model
    img = cv2.imread(filename,0)
    img = cv2.resize(img,(64,64), interpolation = cv2.INTER_CUBIC)
    img = img.reshape(1,64,64,1)
    img = (img-127.0)/127.0
    preds = segmented_model.predict(img)
    preds = preds[0]
    print(preds.shape)
    orig = cv2.imread(filename,0)
    orig = cv2.resize(orig,(300,300),interpolation = cv2.INTER_CUBIC)
    cv2.imwrite("test1.png",orig)
    segmented_image = cv2.resize(preds,(300,300),interpolation =
cv2.INTER_CUBIC)
    cv2.imwrite("myimg.png",segmented_image*255)
    edge_detection = edgeDetection()
    return segmented_image*255, edge_detection

def uploadDataset(): #function to upload dataset
    global filename
    filename = filedialog.askdirectory(initialdir=".")
    text.delete('1.0', END)

```

```

text.insert(END,filename+" loaded\n");

def datasetPreprocessing():
    global X
    global Y
    X.clear()
    Y.clear()
    if os.path.exists('Model/myimg_data.txt.npy'):
        X = np.load('Model/myimg_data.txt.npy')
        Y = np.load('Model/myimg_label.txt.npy')
    else:
        for root, dirs, directory in os.walk(filename+"/no"):
            for i in range(len(directory)):
                name = directory[i]
                img = cv2.imread(filename+"/no/"+name,0) #reading images
                ret2,th2 =
cv.threshold(img,0,255,cv.THRESH_BINARY+cv.THRESH_OTSU)
#processing and normalization images
                img = cv2.resize(img, (128,128)) #resizing images
                im2arr = np.array(img) #extract features from images
                im2arr = im2arr.reshape(128,128,1)
                X.append(im2arr)
                Y.append(0)
                print(filename+"/no/"+name)

        for root, dirs, directory in os.walk(filename+"/yes"):
            for i in range(len(directory)):
                name = directory[i]
                img = cv2.imread(filename+"/yes/"+name,0)
                ret2,th2 =
cv.threshold(img,0,255,cv.THRESH_BINARY+cv.THRESH_OTSU)
                img = cv2.resize(img, (128,128))
                im2arr = np.array(img)
                im2arr = im2arr.reshape(128,128,1)
                X.append(im2arr)
                Y.append(1)
                print(filename+"/yes/"+name)

    X = np.asarray(X)
    Y = np.asarray(Y)
    np.save("Model/myimg_data.txt",X)
    np.save("Model/myimg_label.txt",Y)
    print(X.shape)

```

```

print(Y.shape)
print(Y)
cv2.imshow('ss',X[20])
cv2.waitKey(0)
text.insert(END,"Total number of images found in dataset :
"+str(len(X))+"\n")
text.insert(END,"Total number of classes : "+str(len(set(Y))+"\n\n")
text.insert(END,"Class labels found in dataset : "+str(disease))

def trainTumorDetectionModel():
    global accuracy
    global classifier

    YY = to_categorical(Y)

    indices = np.arange(X.shape[0])
    np.random.shuffle(indices)

    x_train = X[indices]
    y_train = YY[indices]

    if os.path.exists('Model/model.json'):
        with open('Model/model.json', "r") as json_file:
            loaded_model_json = json_file.read()
            classifier = model_from_json(loaded_model_json)

            classifier.load_weights("Model/model_weights.h5")
            classifier._make_predict_function()
    else:
        X_trains, X_tests, y_trains, y_tests = train_test_split(x_train, y_train,
test_size = 0.2, random_state = 0)
        classifier = Sequential()
        classifier.add(Convolution2D(32, 3, 3, input_shape = (128, 128, 1),
activation = 'relu'))
        classifier.add(MaxPooling2D(pool_size = (2, 2)))
        classifier.add(Convolution2D(32, 3, 3, activation = 'relu'))
        classifier.add(MaxPooling2D(pool_size = (2, 2)))
        classifier.add(Flatten())
        classifier.add(Dense(output_dim = 128, activation = 'relu'))
        classifier.add(Dense(output_dim = 2, activation = 'softmax'))
        print(classifier.summary())

```

```

        classifier.compile(optimizer = 'adam', loss = 'categorical_crossentropy',
metrics = ['accuracy'])
        hist = classifier.fit(x_train, y_train, batch_size=16,
epochs=10, validation_split=0.2, shuffle=True, verbose=2)
        classifier.save_weights('Model/model_weights.h5')
        model_json = classifier.to_json()
        with open("Model/model.json", "w") as json_file:
            json_file.write(model_json)
        f = open('Model/history.pckl', 'wb')
        pickle.dump(hist.history, f)
        f.close()
        f = open('Model/history.pckl', 'rb')
        data = pickle.load(f)
        f.close()
        acc = data['accuracy']
        accuracy = acc[4] * 100
        text.insert(END, '\n\nCNN Brain Tumor Model Generated. See black console
to view layers of CNN\n\n')
        text.insert(END, "CNN Brain Tumor Prediction Accuracy on Test Images :
"+str(accuracy)+"\n")

```

```

def tumorClassification():
    filename = filedialog.askopenfilename(initialdir="testImages")
    img = cv2.imread(filename,0)
    img = cv2.resize(img, (128,128))
    im2arr = np.array(img)
    im2arr = im2arr.reshape(1,128,128,1)
    XX = np.asarray(im2arr)

    predicts = classifier.predict(XX)
    print(predicts)
    cls = np.argmax(predicts)
    print(cls)
    if cls == 0:
        img = cv2.imread(filename)
        img = cv2.resize(img, (800,500))
        cv2.putText(img, 'Classification Result : '+disease[cls], (10, 25),
cv2.FONT_HERSHEY_SIMPLEX,0.7, (0, 255, 255), 2)
        cv2.imshow('Classification Result : '+disease[cls], img)
        cv2.waitKey(0)
    if cls == 1:

```



```

        segmented_image, edge_image = tumorSegmentation(filename)
        img = cv2.imread(filename)
        img = cv2.resize(img, (800,500))
        cv2.putText(img, 'Classification Result : '+disease[cls], (10, 25),
cv2.FONT_HERSHEY_SIMPLEX,0.7, (0, 255, 255), 2)
        cv2.imshow('Classification Result : '+disease[cls], img)
        cv2.imshow("Tumor Segmented Image",segmented_image)
        cv2.imshow("Edge Detected Image",edge_image)
        cv2.waitKey(0)

```

```

def graph():
    f = open('model/history.pckl', 'rb')
    data = pickle.load(f)
    f.close()

    accuracy = data['accuracy']
    loss = data['loss']

    plt.figure(figsize=(10,6))
    plt.grid(True)
    plt.xlabel('Training Epoch')
    plt.ylabel('Accuracy/Loss')
    plt.plot(loss, 'ro-', color = 'red')
    plt.plot(accuracy, 'ro-', color = 'green')
    plt.legend(['Loss', 'Accuracy'], loc='upper left')
    plt.title('Brain Tumor CNN Model Training Accuracy & Loss Graph')
    plt.show()

```

```

font = ('times', 16, 'bold')
title = Label(main, text='Identifying Brain Tumor using X-Ray Images')
title.config(bg='darkviolet', fg='gold')
title.config(font=font)
title.config(height=3, width=120)
title.place(x=0,y=5)

```

```

font1 = ('times', 12, 'bold')
text=Text(main,height=20,width=150)
scroll=Scrollbar(text)
text.configure(yscrollcommand=scroll.set)
text.place(x=50,y=120)

```

```
text.config(font=font1)
```

```
font1 = ('times', 12, 'bold')
uploadButton = Button(main, text="Upload Tumor X-Ray Images Dataset",
command=uploadDataset)
uploadButton.place(x=50,y=550)
uploadButton.config(font=font1)
```

```
preprocessButton = Button(main, text="Dataset Preprocessing & Features
Extraction", command=datasetPreprocessing)
preprocessButton.place(x=430,y=550)
preprocessButton.config(font=font1)
```

```
cnnButton = Button(main, text="Trained CNN Brain Tumor Detection Model",
command=trainTumorDetectionModel)
cnnButton.place(x=810,y=550)
cnnButton.config(font=font1)
```

```
classifyButton = Button(main, text="Brain Tumor Segmentation &
Classification", command=tumorClassification)
classifyButton.place(x=50,y=600)
classifyButton.config(font=font1)
```

```
graphButton = Button(main, text="Training Accuracy Graph",
command=graph)
graphButton.place(x=430,y=600)
graphButton.config(font=font1)
```

```
main.config(bg='turquoise')
main.mainloop()
```

CHAPTER-9: RESULTS

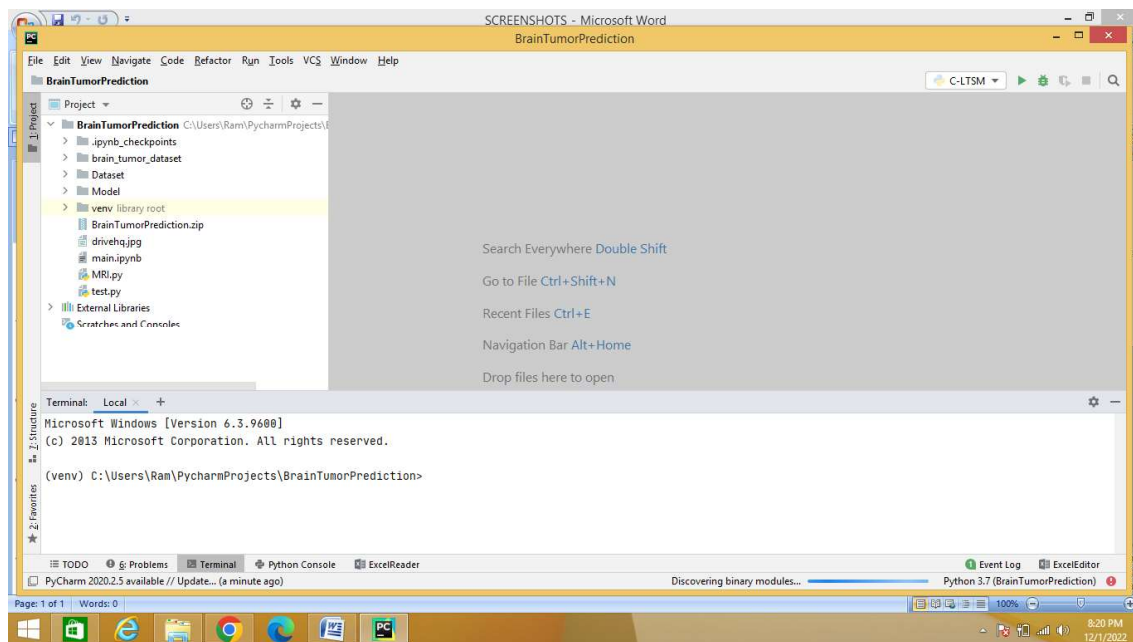


Figure 4: 'Opening the project'

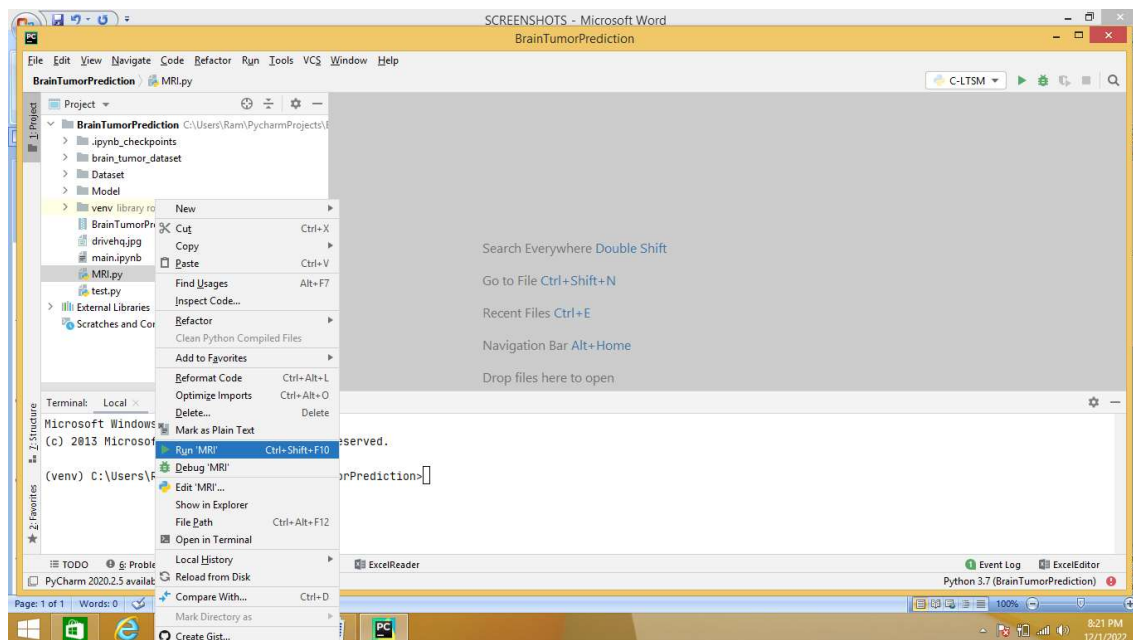


Figure 5: 'Run the Project'

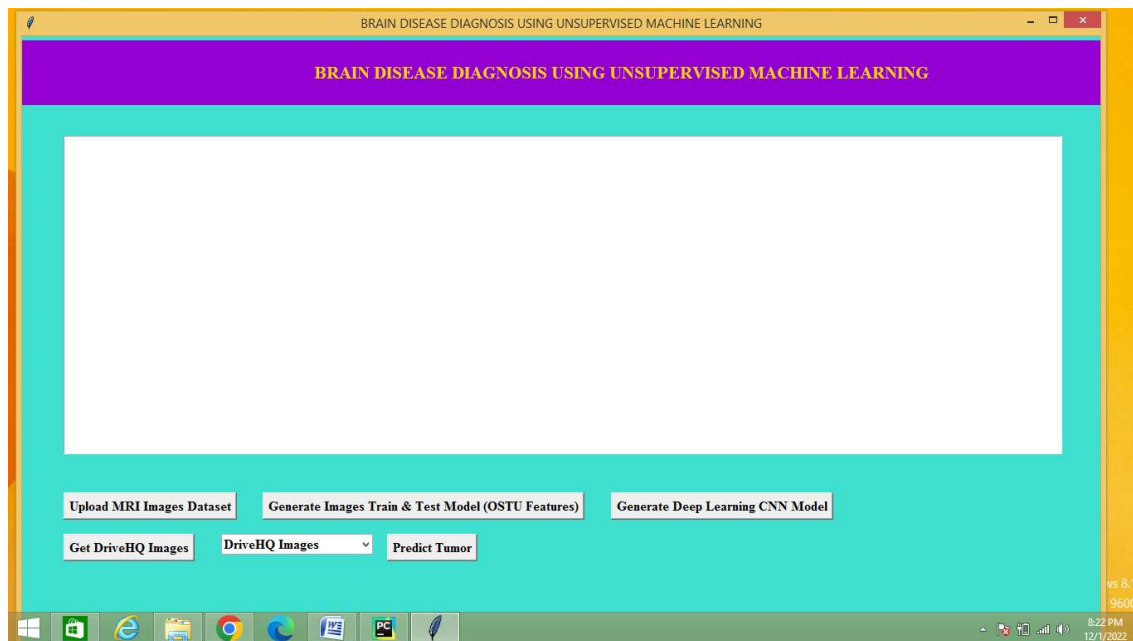


Figure 6: ‘GUI of the Project’

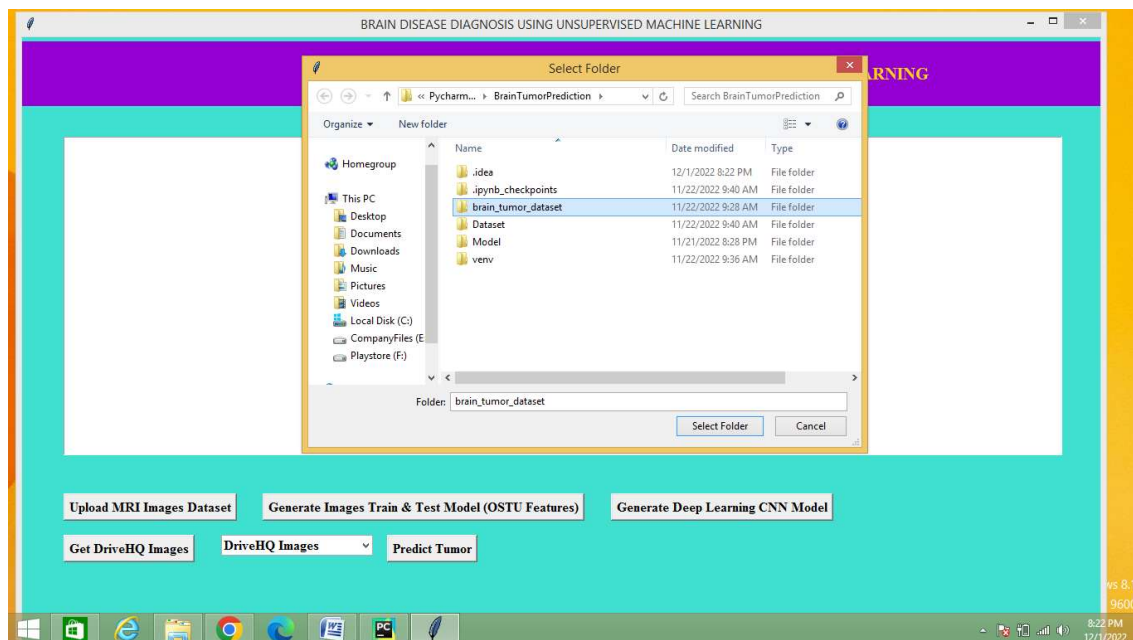


Figure 7: ‘Upload MRI Images Dataset’

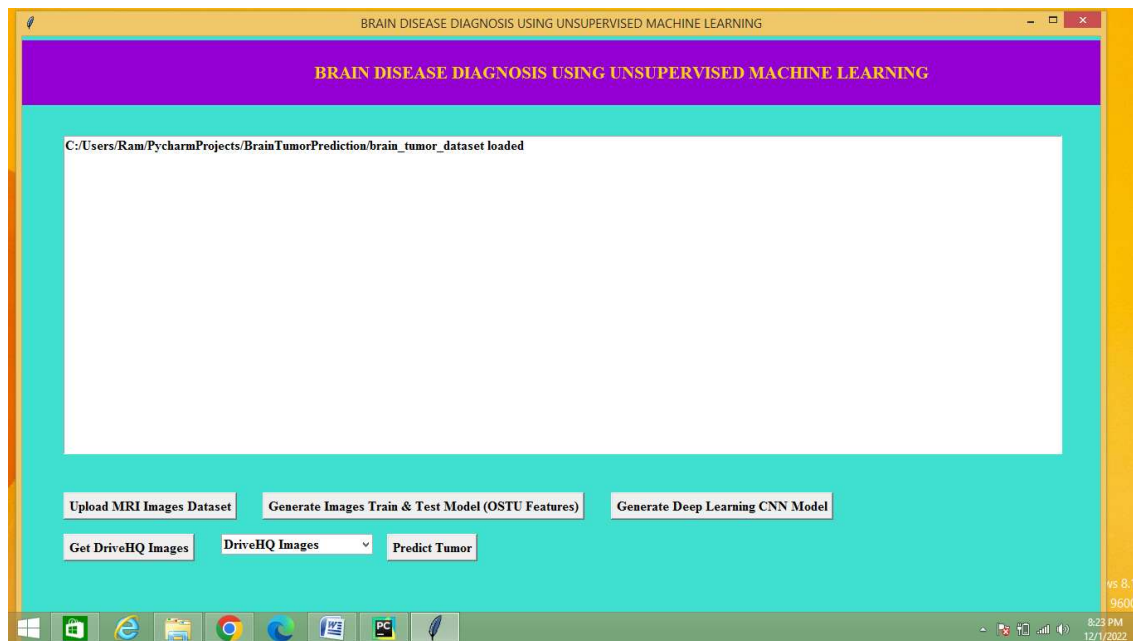


Figure 8: 'Uploaded Dataset Screen'

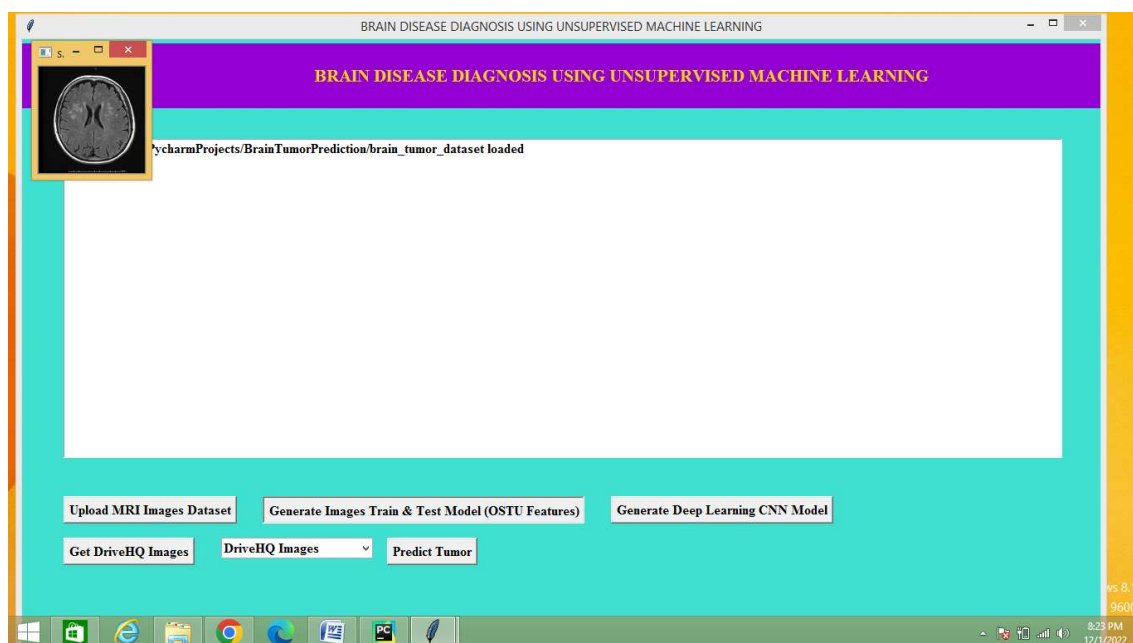


Figure 9: 'Generate Images Train & Test Model'

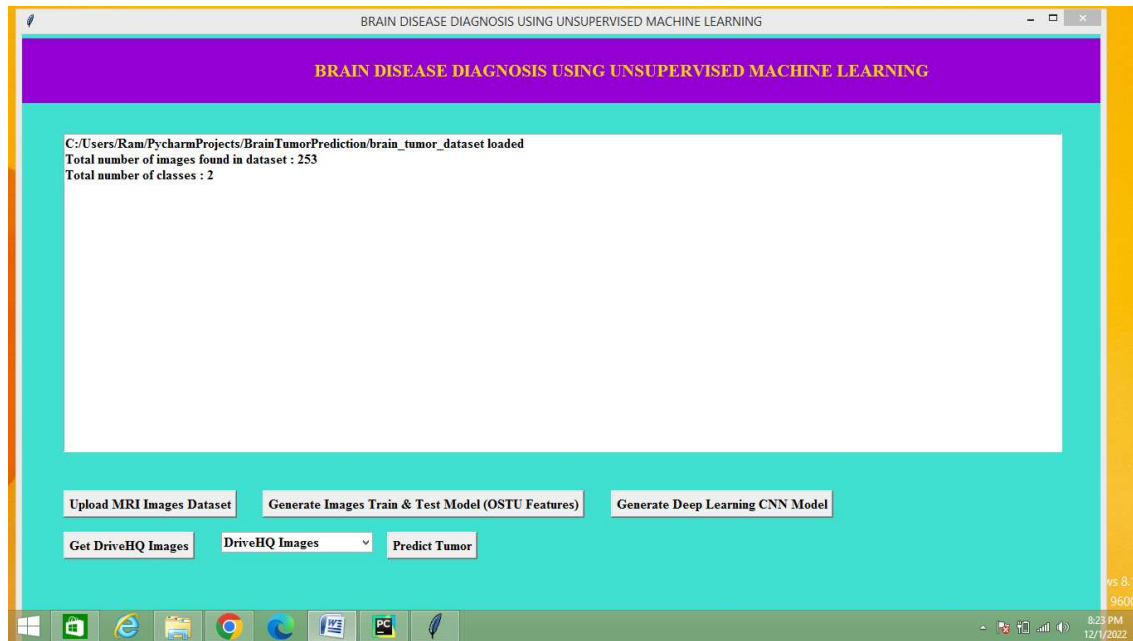


Figure 10: ‘Display of Classes and Images’

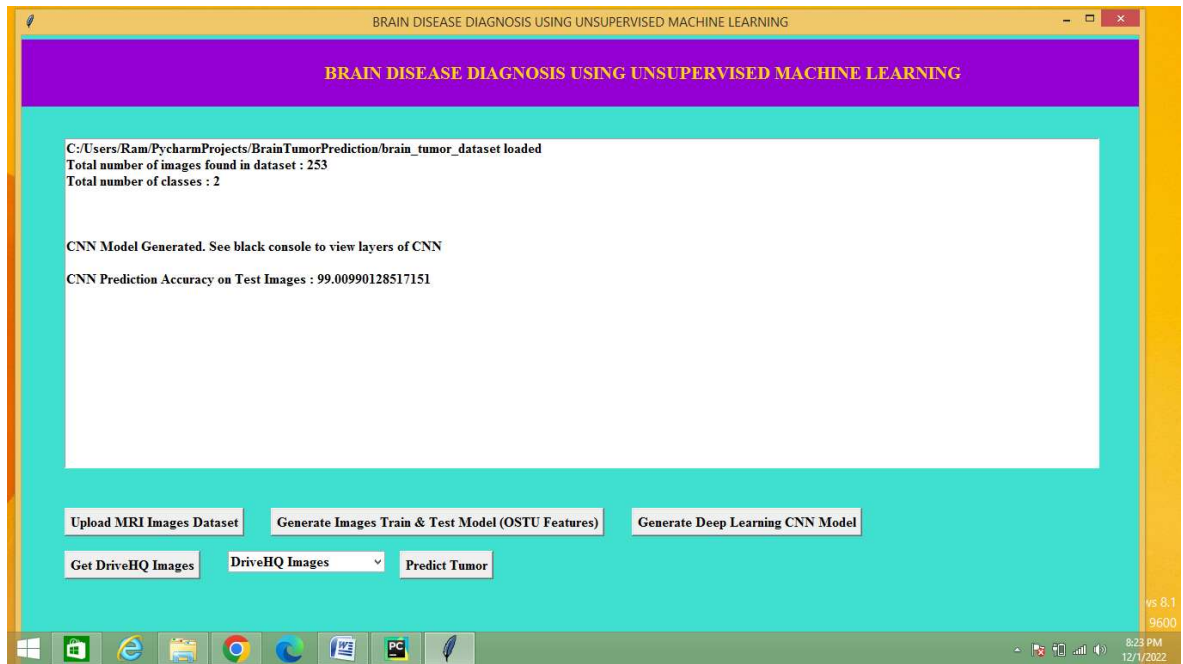


Figure 11: ‘Generate Deep Learning CNN Model’

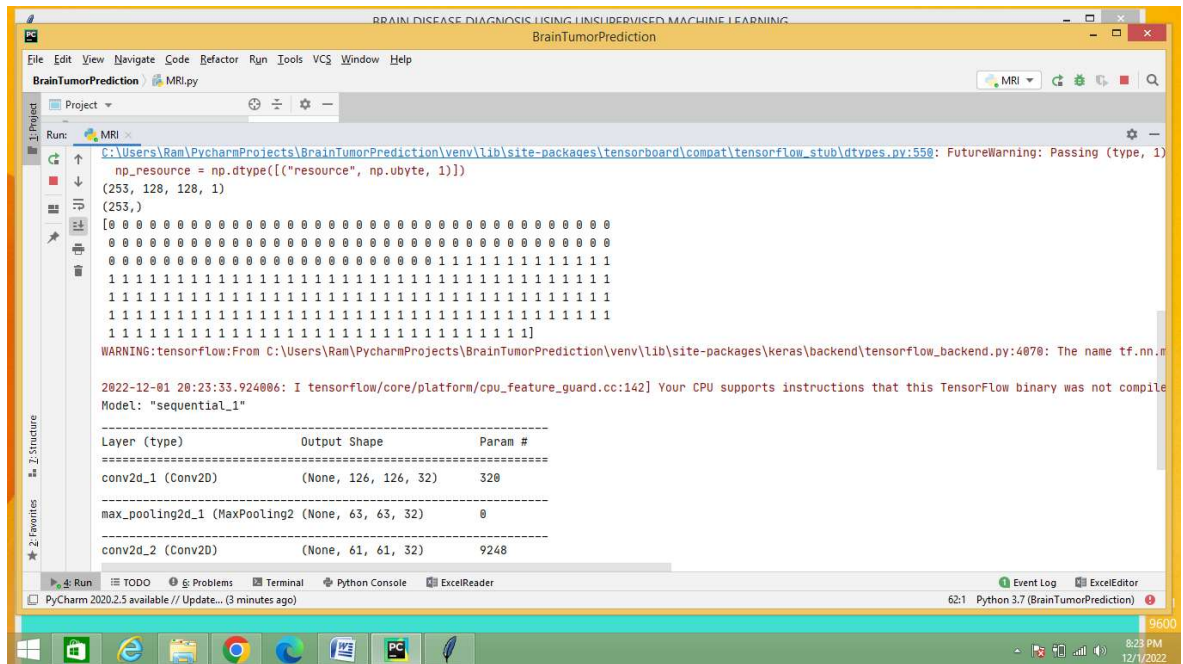


Figure 12: ‘Output of Model Training 1’

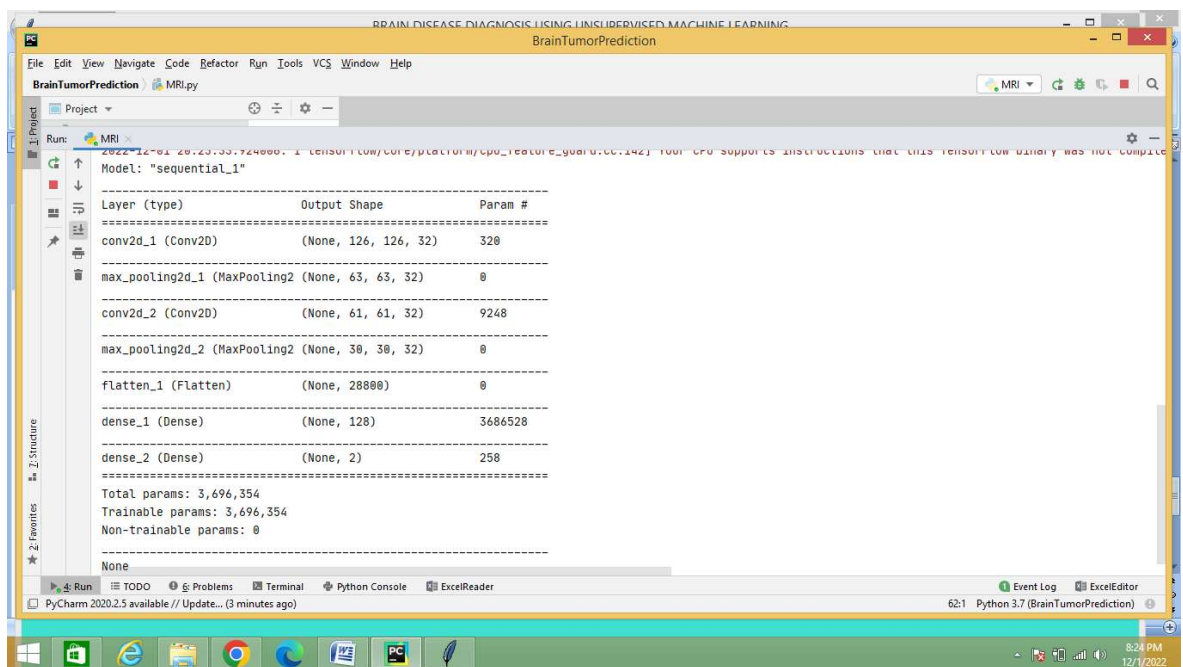


Figure 13: ‘Output of Model Training 2’

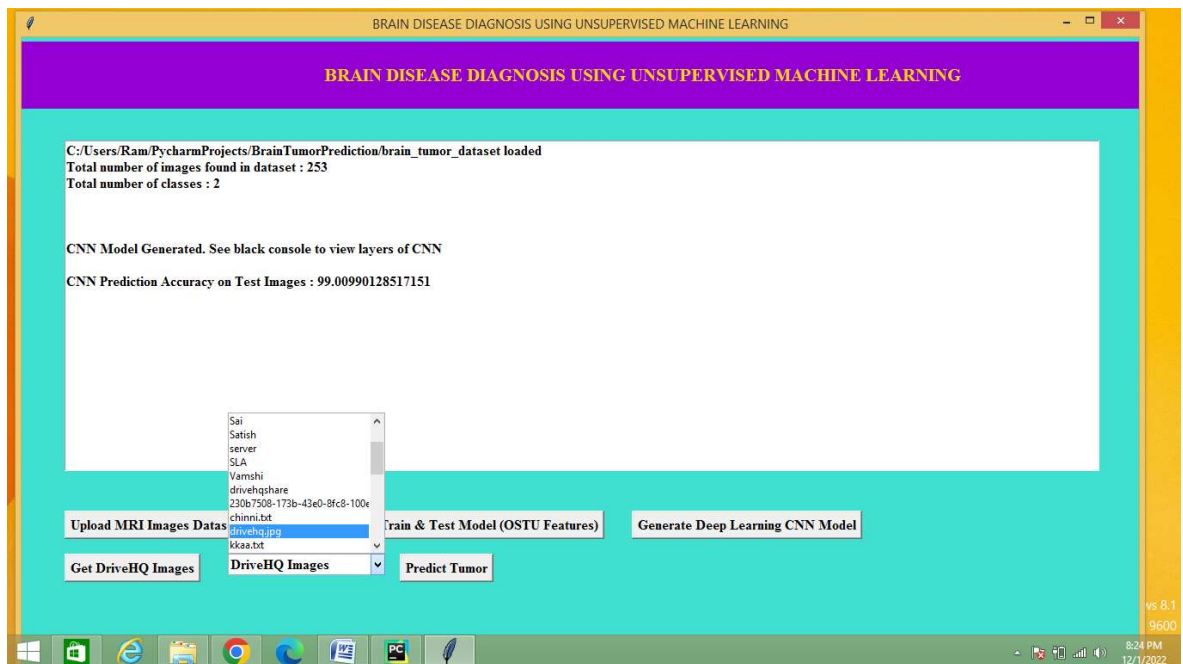


Figure 14: 'Test the Model'

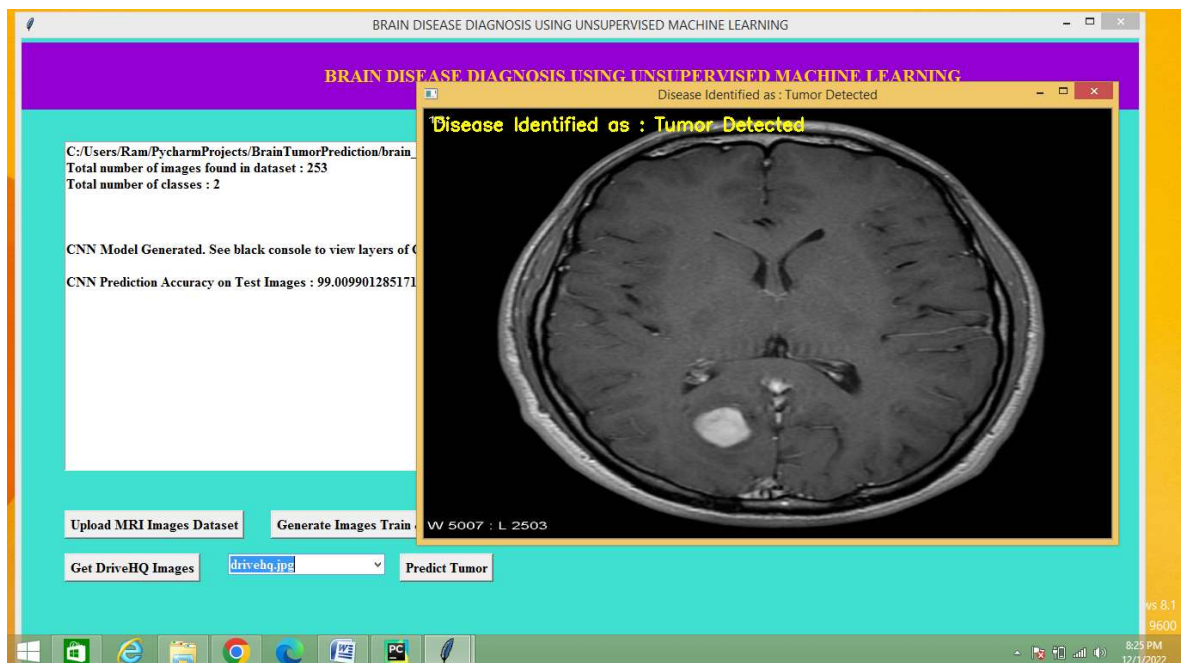


Figure 4: 'Output Screen of Brain Tumor'

CHAPTER-10: CONCLUSION & FUTURE ENHANCEMENT

Conclusion

In this paper, we have presented a survey on the four most dangerous brain disease detection processes using machine and deep learning. The survey reveals some important insights into contemporary ML/DL techniques in the medical field used in today's brain disorder research. With the passage of time, identification, feature extraction, and classification methods are becoming more challenging in the field of ML and DL. Researchers across the globe are working hard to improve these processes by exploring different possible ways. One of the most important factors is to improve classification accuracy. For this, the number of training data needs to be increased because the more the data is involved, the more accurate the results will be. The use of hybrid algorithms and a combination of supervised with unsupervised and ML with DL methods are promising to provide better results. Even, various fine tunings can sometimes offer promising improvements. For example, in [83], 3D-CNN is used first to extract primary features, and next, instead of the general FC layer, the FS Bi-LSTM issued. This slight change in a part of the system eventually resulted in superior performances. Based on the discussion on different types of brain disease data sources and feature extractions methods, it is apparent that the accuracies differ based on different classifiers used and feature extraction processes applied in the systems. To uncover the limitations of existing ML/DL-based approaches to detect various brain diseases, the paper provides a discussion focusing on a set of open research issues. To design effective AI systems for medical applications, the inclusion of XAI approaches is the ultimate necessity. This will help medical professionals to build their confidence and AI-based solutions will be transformed into clinical practice in the treatment of patients with brain disorders. We came to know that quality of training data and interoperability are also major concerns to develop ML and DL-based solutions. It is yet to be determined whether we will be able to have sufficient training data without compromising the performances of DL/ML algorithms. To make ML/DL-based solutions more practical, various other issues such as resource efficiency, large-scale medical data management, and security and privacy should be addressed well. This survey is expected to be useful for researchers working in the area of AI and medical applications in general and ML/DL-based brain disease detection in particular.

Future Enhancement

Advanced Clustering Techniques: Investigate and implement more sophisticated clustering algorithms that can capture complex patterns in brain imaging data. Techniques like hierarchical clustering, density-based spatial clustering of applications with noise (DBSCAN), or advanced variants of Gaussian Mixture Models (GMM) could provide more nuanced insights.

Deep Learning Integration: Incorporate deep learning models, such as Convolutional Neural Networks (CNNs) or Autoencoders, to preprocess and extract high-level features from brain images. These features can then be used as input for your unsupervised learning algorithms, potentially improving the accuracy and robustness of your diagnoses.

Multi-Modal Data Fusion: Combine data from multiple imaging modalities (e.g., MRI, PET, CT) and non-imaging data (e.g., genetic information, clinical records) to create a more comprehensive representation of brain diseases. Unsupervised learning techniques that can handle heterogeneous data sources can provide a more holistic view of the disease.

Anomaly Detection: Focus on anomaly detection techniques that can identify rare or novel patterns indicative of brain diseases. Unsupervised learning methods like One-Class SVM or Isolation Forest can be particularly useful for detecting outliers that may represent early signs of disease.

Explainability and Interpretability: Enhance the interpretability of your unsupervised learning models by incorporating techniques that provide insights into the decision-making process. Tools like SHAP (SHapley Additive exPlanations) or LIME (Local Interpretable Model-agnostic Explanations) can help clinicians understand and trust the model's outputs.

Temporal Analysis: Incorporate time-series analysis to study the progression of brain diseases over time. Unsupervised learning methods that can handle sequential data, such as Hidden Markov Models or Recurrent Neural Networks, can help in predicting disease trajectories and identifying critical stages.

Transfer Learning: Utilize transfer learning to leverage pre-trained models on large datasets for brain imaging analysis. This can help in overcoming the limitations of small datasets and improve the generalizability of your models.

Personalized Medicine: Develop models that can adapt to individual patient characteristics, such as age, genetic profile, and lifestyle factors. Unsupervised learning techniques that can personalize the diagnosis process can lead to more tailored and effective treatment plans.

Real-Time Diagnosis: Work towards real-time or near-real-time diagnosis by optimizing your models for speed and efficiency. This can be particularly valuable in clinical settings where quick and accurate diagnoses are crucial.

Interdisciplinary Collaboration: Collaborate with experts in neurology, radiology, and bioinformatics to ensure that your models are clinically relevant and aligned with current medical practices. This interdisciplinary approach can lead to more impactful and practical applications of your research.

CHAPTER 11: REFERENCE

1. Anilkumar, B. and P. Rajesh Kumar, "Multi Tumor Classification in MR Brain Images through deep Feature Extraction using CNN and Supervised Classifier", International Journal of Emerging Technologies, Vol.11, No.1, PP: 83–90, 2020.
2. S. M. Shelke and S. W. Mohod, "Automated Segmentation and Detection of Brain Tumor from MRI," 2018 International Conference on Advances in Computing, Communications and Informatics (ICACCI), Bangalore, pp. 2120-2126, 2018, doi: 10.1109/ICACCI.2018.8554807.
3. A.Hebli and S. Gupta, "Brain tumor prediction and classification using support vector machine," 2017 International Conference on Advances in Computing, Communication and Control (ICAC3), Mumbai, pp. 1-6, 2017.
4. S. R. Telrandhe, A. Pimpalkar and A. Kendhe, "Detection of brain tumor from MRI images by using segmentation & SVM," 2016 World Conference on Futuristic Trends in Research and Innovation for Social Welfare (Startup Conclave), Coimbatore, pp. 1-6,2016,doi: 10.1109/STARTUP.2016.7583949.
5. Ayachi R., Ben Amor N. "Brain Tumor Segmentation Using Support Vector Machines", In: Sossai C., Chemello G. (eds) Symbolic and Quantitative Approaches to Reasoning with Uncertainty. ECSQARU 2009. Lecture Notes in Computer Science, vol 5590. Springer, Berlin, Heidelberg, 2009.

CHAPTER 11: YUKTHI INNOVATION CERTIFICATE

		INSTITUTION'S INNOVATION COUNCIL MOE'S INNOVATION CELL			
Institute Name:					
Malla Reddy Institute of Technology & Science					
Title of the Innovation/Prototype:					
Deep Learning Applications In Medical Image Analysis-Brain Tumor					
Team Lead Name:	Team Lead Email:	Team Lead Phone:	Team Lead Gender:		
NANDIKONDA LIKHITH REDDY	likithreddy123@gmail.com	8555032453	Male		
FY of Development:	Developed as part of:	Innovation Type:	TRL LEVEL:		
2023-24	Academic Requirement/Study Project	Process, Service	4		
MRL Level:					
MRL 1: Basic manufacturing implications identified					
IRL Level:					
IRL 1: Basic Research (Need Identification & Peer Review Publications) & Completed First-Pass Business Model Canvas (BMC)					
Theme:					
Healthcare & Biomedical devices.,					
Define the problem and its relevance to today's market / society / industry need:					
Our project addresses two critical issues in medical diagnostics: brain cancer detection using X-ray/CT scans and epilepsy seizure prediction using EEG signals. By leveraging advanced deep learning techniques, such as MobileNetV2 for image preprocessing and a CNN integrated with C-LSTM for EEG analysis, our system aims to enhance the accuracy and efficiency of diagnosing brain cancer and predicting epileptic seizures. This technology has the potential to significantly improve patient outcomes by enabling earlier detection and more precise monitoring, addressing a crucial need in today's healthcare industry.					
Describe the Solution / Proposed / Developed:					
Our solution integrates advanced AI methodologies to improve medical diagnostics. For brain cancer detection, we preprocess X-ray/CT scans using MobileNetV2 and hot encoding, resizing images to 256×256 pixels for optimal model performance. For epilepsy detection, we utilize EEG recordings, applying a novel CNN combined with a C-LSTM approach to predict seizure segments accurately. The data is divided into 70% training, 20% validation, and 10% testing sets to ensure robust model training and evaluation. This comprehensive system aims to enhance early diagnosis and precise monitoring, addressing critical healthcare challenges.					
Explain the uniqueness and distinctive features of the (product / process / service) solution:					
Our solution stands out due to its dual focus on brain cancer and epilepsy detection using state-of-the-art AI techniques. The unique integration of MobileNetV2 for X-ray/CT scan preprocessing and a hybrid CNN-C-LSTM model for EEG analysis ensures high accuracy and efficiency. The resizing of images to 256×256 pixels optimizes model performance, while hot encoding facilitates accurate label processing. Additionally, the structured data partitioning into training, validation, and testing sets enhances model reliability. This innovative approach significantly improves early diagnosis and precise monitoring, offering a distinctive and impactful advancement in medical diagnostics.					

How your proposed / developed (product / process / service) solution is different from similar kind of product by the competitors if any: Our solution differentiates itself by combining two advanced AI-driven diagnostic tools for both brain cancer detection and epilepsy seizure prediction. Unlike competitors, we employ MobileNetV2 for effective preprocessing of X-ray/CT scans and a unique CNN-C-LSTM hybrid model for analyzing EEG data. This dual-focus approach ensures comprehensive, high-accuracy diagnostics. Additionally, the meticulous preprocessing steps, including image resizing to 256×256 pixels and hot encoding of labels, enhance model performance. Our structured data partitioning strategy further ensures reliable outcomes, providing a distinct, holistic solution that addresses multiple critical healthcare needs more effectively than existing products.
Is there any IP or Patentable Component associated with the Solution?: No
Has the Solution Received any Innovation Grant/Seedfund Support?: No
Are there any Recognitions (National/International) Obtained by the Solution?: No
*Is the Solution Commercialized either through Technology Transfer or Enterprise Development/Startup?: No
Had the Solution Received any Pre-Incubation/Incubation Support?: No
Video URL: https://docs.google.com/presentation/d/1OK51x2V0iAjDg9I_nNNrBqZaMs5VaDZD/edit#slide=id.p1
Innovation Photograph: View File
This report is electronically generated against Yukti - National Innovation Repository Portal. Downloaded on: 09-08-2024

Add Team Member Details

Team Member Details						
Name	Email	Phone	Designation	Gender	Caste	Action
ADEM VINAY	ademvinay2207@gmail.com	8522933422	Student	Male	NON ST	Edit Delete
THANUGULA MANOHAR	tanugulamanohar62789@gmail.com	7330783365	Student	Male	NON ST	Edit Delete
DEVADIGA ANIKETH	anikethdev21@gmail.com	9505970052	Student	Male	NON ST	Edit Delete

Add Mentor Details

Team Mentor Details					
Name	Email	Phone	Designation	Organization	Action
Dr. VAKA MURALI MOHAN	vakamuralimohan@gmail.com	9966023477	Principal & Professor of CSE	MALLA REDDY INSTITUTE OF TECHNOLOGY AND SCIENCE	Edit Delete